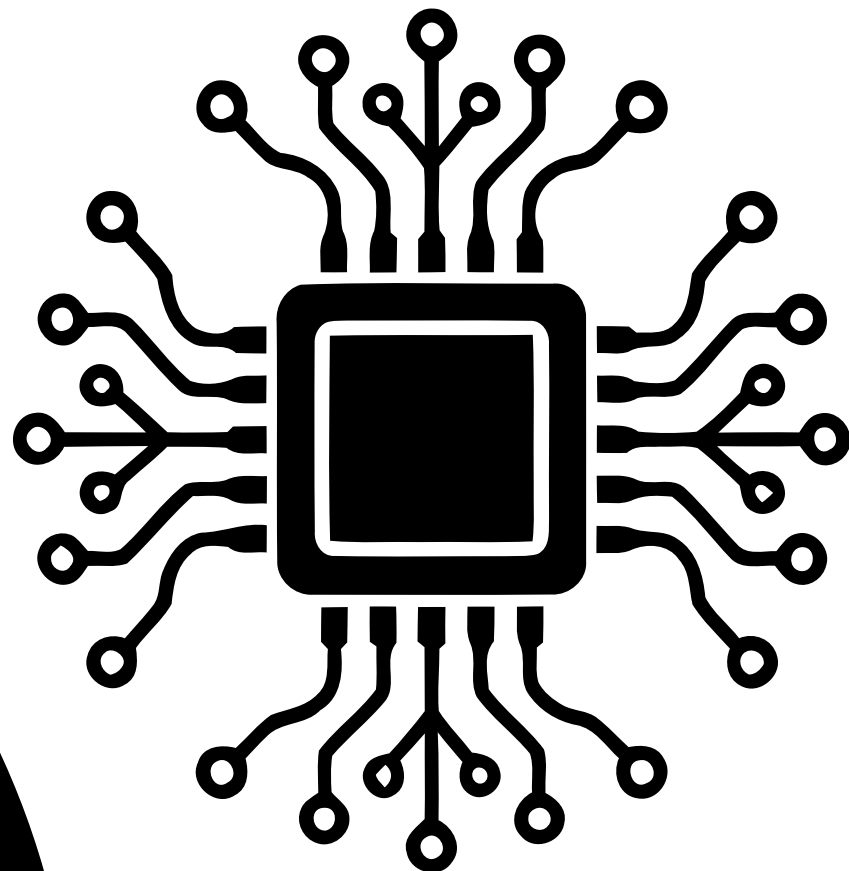
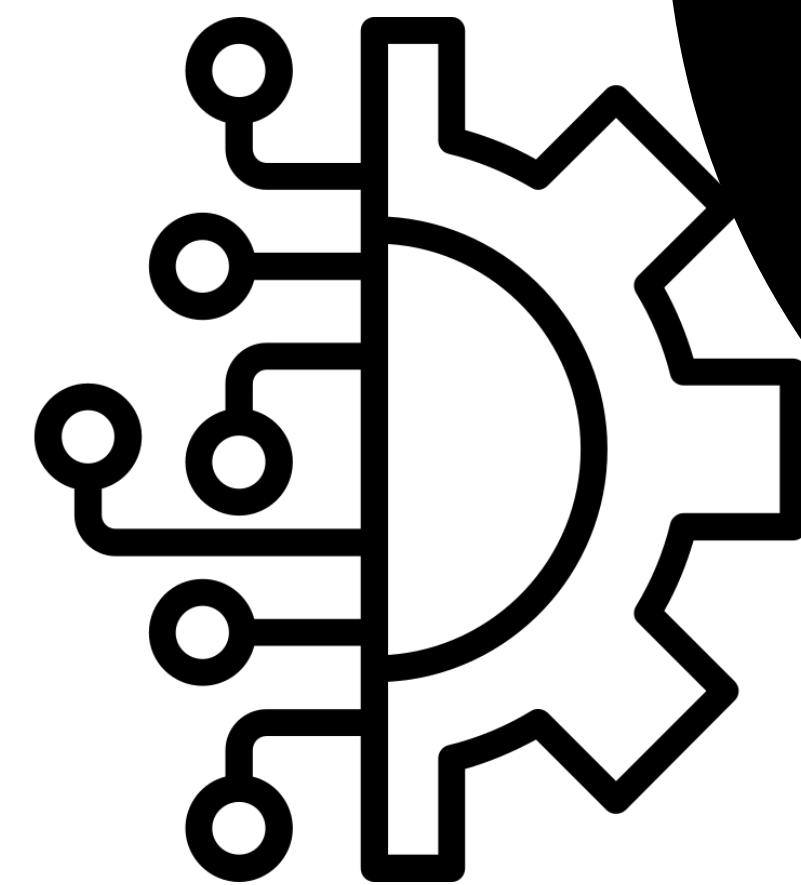




# SEMINARIO

COMPARAÇÃO ENTRE  
PYTEST E UNITTEST



# O QUE SÃO TESTES UNITÁRIOS?

Testes unitários são testes utilizados para verificar se **pequenas partes de um programa funcionam corretamente**.

Por exemplo, suponha que temos uma função:

```
def somar(a, b):  
    return a + b
```

Podemos criar um teste para verificar se:

```
somar(2, 3) = 5
```



# PYTEST



pytest

O Pytest é um framework de testes para Python conhecido por permitir escrever testes de maneira simples e objetiva.

## Exemplo

Arquivo calculadora.py:

```
def somar(a, b): return a + b
def subtrair(a, b):
    return a - b
```

```
from calculadora import somar, subtrair
def test_somar(): assert somar(2, 3) == 5
def test_subtrair(): assert subtrair(10, 4) == 6
```

## Uma saída possível será:

```
test session starts
                        collected 2 items test_calculadora.py ..
[100%] 2 passed in 0.02s
```

-  PONTOS POSITIVOS DO PYTEST
- 
- SINTAXE SIMPLES.
- UTILIZA ASSERT DIRETAMENTE.
- FÁCIL DE APRENDER.
- POSSUI MUITOS RECURSOS.
- PERMITE ORGANIZAR TESTES DE MANEIRA FLEXÍVEL.
- POSSUI PLUGINS PARA AMPLIAR SUAS FUNCIONALIDADES.
- A SAÍDA DOS TESTES É BASTANTE CLARA.
- 
-  PONTOS NEGATIVOS
- 
- É NECESSÁRIO INSTALAR UMA BIBLIOTECA ADICIONAL.
- PARA QUEM ESTÁ ACOSTUMADO EXCLUSIVAMENTE COM UNITTEST, ALGUNS RECURSOS PODEM PARECER DIFERENTES.

# UNITTEST

O **Unittest** é o framework de testes que já faz parte da biblioteca padrão do Python. Portanto, normalmente **não é necessário instalar nada adicional**.



```
import unittest
from calculadora import somar, subtrair

class TestCalculadora(unittest.TestCase):
    def test_somar(self):
        self.assertEqual(somar(2, 3), 5)
    def test_subtrair(self):
        self.assertEqual(subtrair(10, 4), 6)
if __name__ == '__main__': unittest.main()
```

Ran 2 tests in 0.001s

OK

✓ Pontos positivos do Unittest

- Já vem com o Python.
- Não precisa instalar uma biblioteca externa.
- É baseado em uma estrutura organizada de classes.
- Possui diversos métodos de verificação, como `assertEqual`, `assertTrue` e `assertRaises`.
- É bastante utilizado e consolidado.

✗ Pontos negativos

- Possui uma sintaxe mais verbosa.
- Normalmente exige a criação de classes.
- Os testes podem ficar maiores que os equivalentes em Pytest.
- A escrita pode ser menos intuitiva para iniciantes.

# COMPARAÇÃO

| Característica       | Pytest                                    | Unittest                        |
|----------------------|-------------------------------------------|---------------------------------|
| Linguagem            | Python                                    | Python                          |
| Instalação           | Precisa instalar                          | Já vem com Python               |
| Sintaxe              | Mais simples                              | Mais verbosa                    |
| Classes obrigatórias | Não                                       | Geralmente sim                  |
| <code>assert</code>  | <code>assert resultado == esperado</code> | <code>self.assertEqual()</code> |
| Facilidade           | ★★★★★                                     | ★★★★                            |
| Flexibilidade        | ★★★★★                                     | ★★★★                            |
| Organização          | Excelente                                 | Excelente                       |
| Plugins              | Muitos                                    | Mais limitado                   |
| Curva de aprendizado | Baixa                                     | Média                           |



# DIFERENÇA DE IMPLEMENTAÇÃO

```
def test_somar(): assert somar(2, 3) == 5
```

```
class TestCalculadora(unittest.TestCase):  
    def test_somar(self):  
        self.assertEqual(somar(2, 3), 5)
```

- **importar unittest;**
- **criar uma classe;**
- **herdar de unittest.TestCase;**
- **criar o método de teste;**
- **utilizar self.assertEqual().**

**Conclusão: o Pytest consegue realizar o mesmo teste com menos código.**



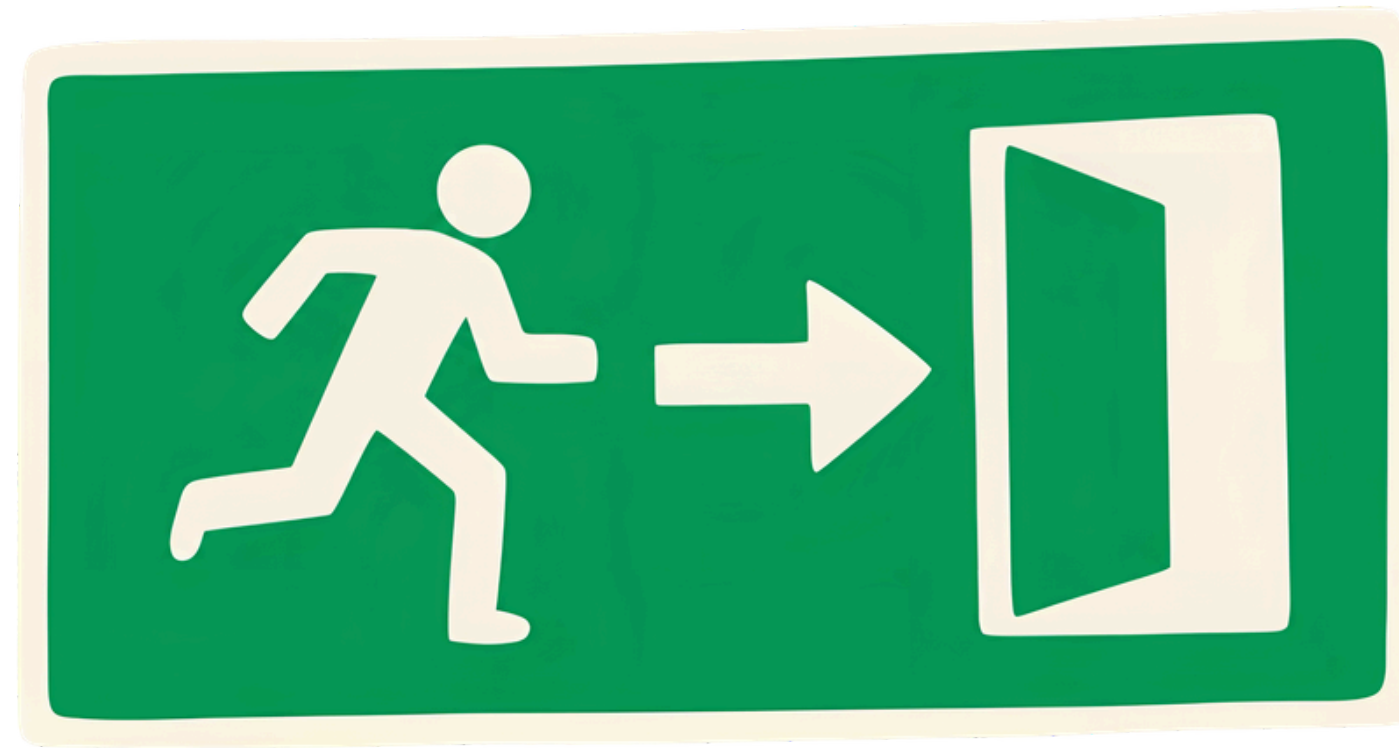
# DIFERENÇA NA SAÍDA

## Pytest

test\_calculadora.py .. [100%] 2 passed in 0.02s

## Unittest

Ran 2 tests in 0.001s OK





# CONCLUSÃO

Após comparar Pytest e Unittest, podemos concluir que os dois frameworks são capazes de realizar testes unitários em Python de forma eficiente. O Unittest possui a vantagem de já fazer parte da biblioteca padrão do Python e apresenta uma estrutura bastante organizada. Por outro lado, o Pytest apresenta uma sintaxe mais simples, menor quantidade de código e grande flexibilidade. Portanto, para projetos que priorizam simplicidade e produtividade, o Pytest pode ser uma opção mais interessante, enquanto o Unittest é uma alternativa sólida para quem deseja utilizar apenas recursos nativos do Python



**TEST**



**OBRIGADO**

**TRABALHO FEITO POR:**

**DIOGO MARX LOPES DOS SANTOS**

**RA:325132950**